Multiple Components

Send data to Component

- Data to a component can be sent as an attribute in the template

```
<app-display   [name]= “myName”>  </app-display>
```

- Child component should have input property to use this value
- Input property is provided with @Input() decorator

```
export class  DisplayComponent {
 @Input()  name:string;
}
```

Output binding

- Output binding done through event binding.
- the attribute is of type EventEmitter
- child element can emit an event and bind that event to output

```
export class DisplayComponent {
  @Output() change = new EventEmitter();

  clicked(){
     this.change.emit();
  }
}
```

- Usage:

```
<app-display  (change)='dataChanged()'></app-display>
```

function in parent

Passing data in Output

- emit() method can have some data as argument
- This is linked to $event object of the output event.
- **$event** - Is an expression exposed in event bindings by Angular, which has the value of the event's payload
- $event represents the data sent by emit()
- It is not same $event in normal javascript
- Example: (child component)

```
export class  DisplayComponent {
@Output() change = new EventEmitter();
 email:string;

 send(){
        this.change.emit(this.email);
  }
}
```

Passing data in Output

- Example: (child component template)

```
<input [(email)] />
<button (click)="send()">Click</button>
```

- Example: (parent component template)

```
<app-display  (change)='dataChanged($event)'></app-display>
```

Passing data in Output

- Example: (parent component)

```
export class AppComponent {

  dataChanged(email){
    console.log("Email of person now is "+ email);
  }
}
```

ng-content

- Ng-content object can be created in child object to put content coming from parent
- Use select=” ” as a CSS selector
- Parent will create such elements while using child components
- Selector is matched to place the content of the parent

ng-content - Example

Child Component

```
<div>
  <div >
    <ng-content  select='.heading'></ng-content>
  </div>
  <div>
    <ng-content  select=‘.body'></ng-content>
  </div>
</div>
```

ng-content - Example

Parent Component

```
<app-child>
   <div class='heading'>This is Heading</div>
   <div class="body">
      <h1>line1</h1>
      <h2>line2</h2>
   </div>
</app-child>
```

ng-content - Example

Child Component with content sent by parent

```
<div>
  <div  >
      <div  class="heading" >This is Heading</div>
  </div>
  <div>
     <div  class="body">
        <h1>line1</h1>
        <h2>line2</h2>
     </div>
</div>
```

@ViewChild / @ViewChildren

- This decorator used to get reference of any dom element or child component
- Using the reference variable, the element properties or methods can be accessed and manipulated
- @ViewChild refers the first matching element
- @ViewChildren refers all matching elements as QueryList array

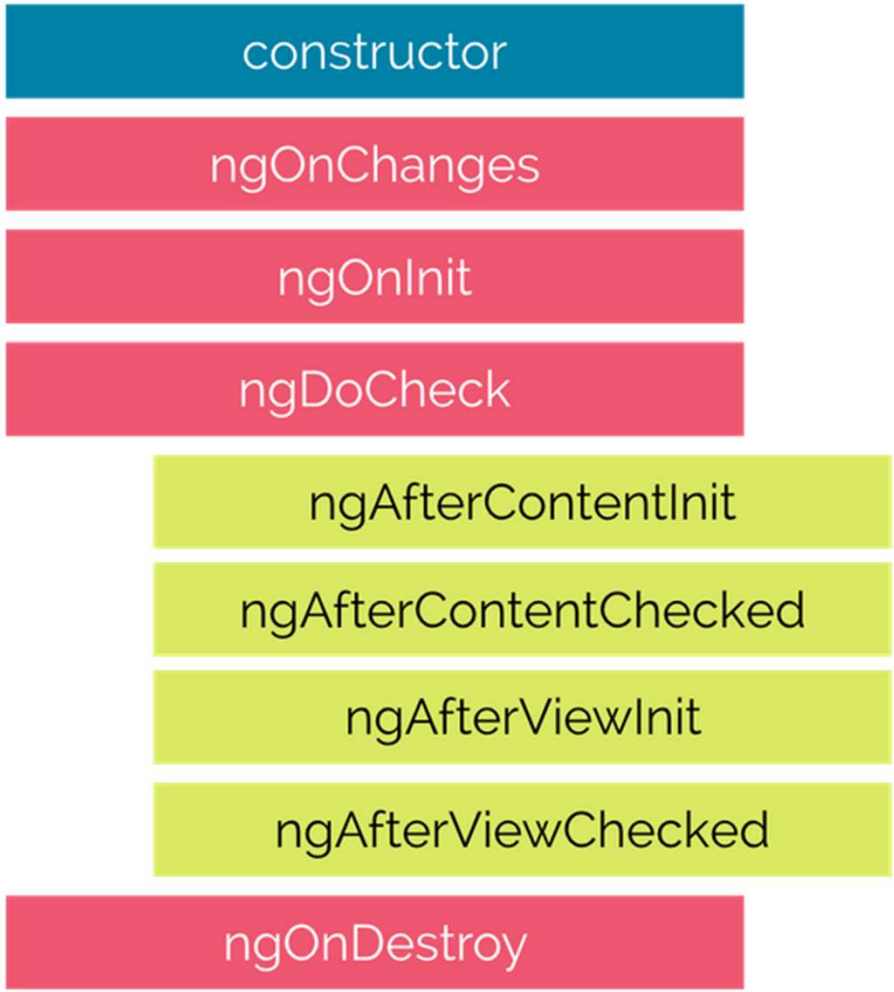
@ContentChild / @ContentChildren

- Used for accessing content received by paren as ng-content

Component lifecycle

- A component in Angular has a life-cycle, a number of different phases it goes through from birth to death.
- We can hook into those different phases to get some pretty fine grained control of our application.
- To do this we add some specific methods to our component class which get called during each of these life-cycle phases, we call those methods hooks.
- We can create methods to execute at every phase

Component lifecycle phases



Component lifecycle

constructor

This is invoked when Angular creates a component or directive by calling new on the class.

ngOnChanges

Invoked every time there is a change in one of th input properties of the component.

ngOnInit

Invoked when given component has been initialized.

This hook is only called once after the first ngOnChanges

ngDoCheck

Invoked when the change detector of the given component is invoked. It allows us to implement our own change detection algorithm for the given component.

ngOnDestroy

This method will be invoked just before Angular destroys the component.

Use this hook to unsubscribe observables and detach event handlers to avoid memory leaks.

Component lifecycle

ngAfterContentInit

Invoked after Angular performs any content projection into the component's view .

ngAfterContentChecked

Invoked each time the content of the given component has been checked by the change detection mechanism of Angular.

ngAfterViewInit

Invoked when the component's view has been fully initialized.

ngAfterViewChecked

Invoked each time the view of the given component has been checked by the change detection mechanism of Angular.

Change Detection Strategies

- Change detection in Angular is executed when:
  - There are updates to Angular State variable
  - Events are invoked inside Angular Component
  - @Input Values are updated for the Components

- Change Detection Strategies are the methods by which the updates to the component is tracked and component is triggered to Re-render
- There are majorly 2 Change Detection Strategies
  - Default Strategy ← this is default
  - onPush Strategy

Implementing onPush strategy

```
@Component({
  selector: "child-component",
  template: `
    <div>
      <h2>{{ this.executeFunction() }}</h2>
    </div>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
```

Dynamic Components

- Dynamic means, that the components location in the application is not defined at buildtime.
- That means, that it is not used in any angular template.
- Instead, the component is instantiated and placed in the application at runtime
- To load dynamic component, first we need to create a container and at runtime get ViewContainerRef through ViewChild

Angular Forms

Bootstrap for Angular

- For using bootstrap in Angular, module can be installed

  ng add @ng-bootstrap/schematics

- bootstrap entry will be added to package.json

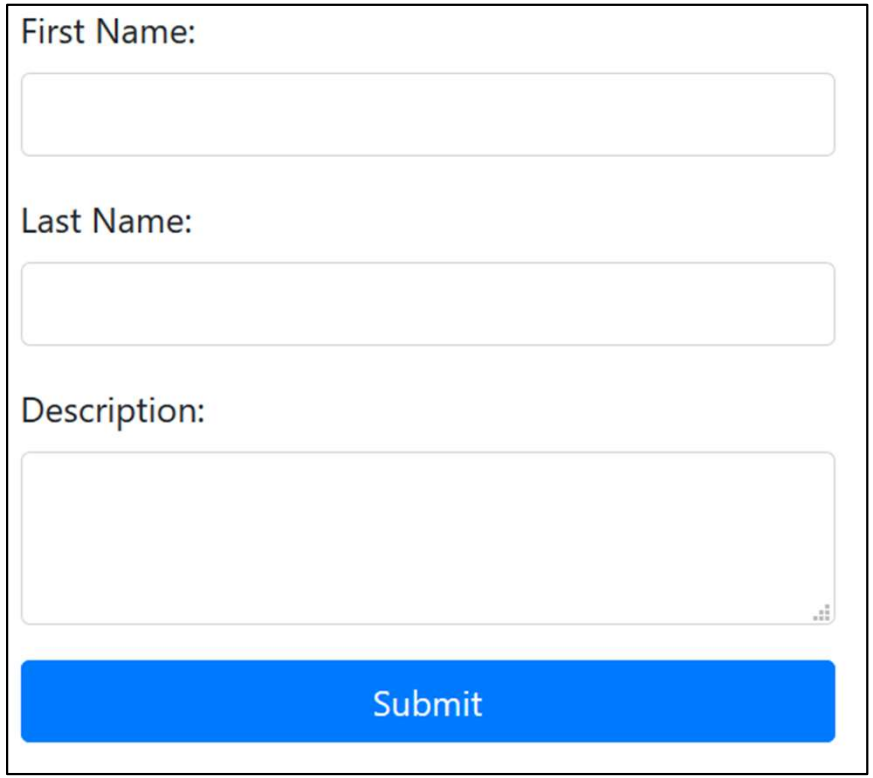
- Use bootstrap classes in templates

```
<div class='form-group' style="width:400px">
    <label for='input1'>Enter id</label>
    <input class='form-control'  id='input1'/>
    <label for="input2">Enter Name</label>
    <input class='form-control'  id='input2'/>
  <div>
```

Bootstrap for Angular

Normal Form

Form using Bootstrap



Angular Form Control

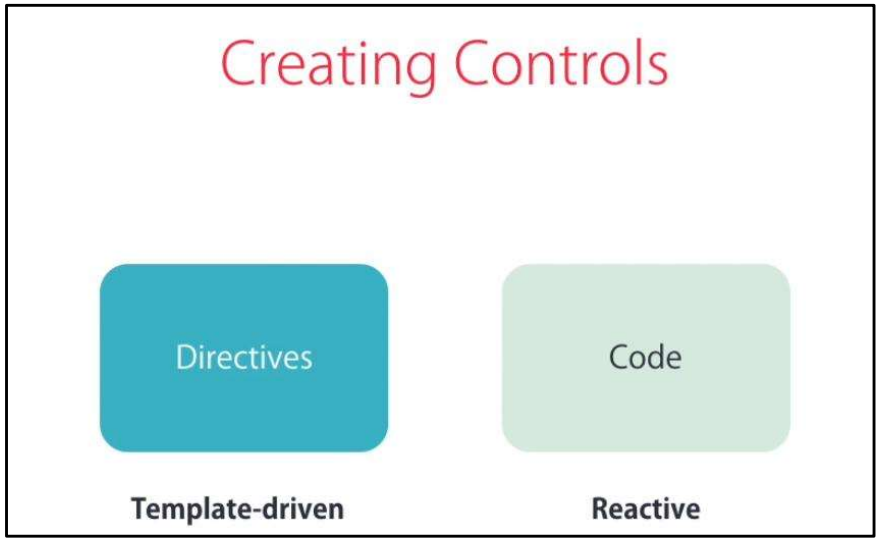
- Following classes used for Form Control
  - FormControl: Associated with each input field
  - FormGroup: associated with multiple input fields
- FormControl has properties:
  - value — value of the input field
  - touched — boolean : true if the field is touched
  - untouched — boolean : true if the field is untouched
  - dirty — boolean : true if the value is changed
  - pristine — boolean : true if the value is not changed
  - valid — boolean : true if field passes all validations
  - errors — provides all the validation errors
- FormGroup has all above properties with respect the group

Angular Form Control

- For form control and validations add FormControl object to each input field and FormGroup object to the form

Creating Controls

- **Template Driven Forms**
  - Controls created by applying Directives
  - Angular creates the control objects implicitly
  - Good for simple forms with basic validations like
    - required
    - range of values etc
- **Reactive Forms**
  - Controls created using custom code
  - Give more control on validation logic
  - Good for complex forms
  - Also called as Model driven Forms



Template Driven Forms

Template Driven Approach

- Teplate Driven forms use ngModel directive

- Input fields should be associated with :-
  - ngModel directive
  - name attribute
  - Template variable

ngModel

- With this directive angular creates all associated controls for the input field
- name attribute is required to associate the ngModel

```
<input   ngModel  name='courseName'  />
```

- To refer the input field and its associated control properties in the template, we can create template variable for ngModel

```
<input  ngModel name='courseName'   #course="ngModel"/>
```

- Template variable and name attribute are two different things

More on ngModel

- Value of the name attribute is the name of the property to represent the value of the field
- Template variable is an object of ngModel and used to refer the field for validations etc

```
<form  #form="ngForm">
   <label for="fn">First Name</label>
   <input id='fn' type="text" ngModel name='fristName'  #nameField='ngModel' />
   <label for="age">Age</label>
   <input id='age' type='text' ngModel name='age'   #ageField='ngModel' />

   <div>{{form.value|json}}</div>
</form>
```

First Name

Sudhir

Age

35

{ "fristName": "Sudhir", "age": "35" }

Validation

- Basic validation supported in HTML5 can be done with ngModel
- ngModel object has properties like valid, touched, dirty, pristine, invalid, errors etc
- Example:

```
<div class='holder'>
  <div class='form-group'>
    <label for="name">Name</label>
    <input id='name' ngModel name='personName' required minlength='5' maxlength='20'
          class="form-control" #name='ngModel'  />
    <div class='alert alert-danger' *ngIf='name.touched && name.invalid'>
      <div *ngIf='name.errors.required'>Name required</div>
      <div *ngIf='name.errors.minlength'>
        Minimum {{name.errors.minlength.requiredLength}} characters required
      </div>
    </div>
  </div>
```

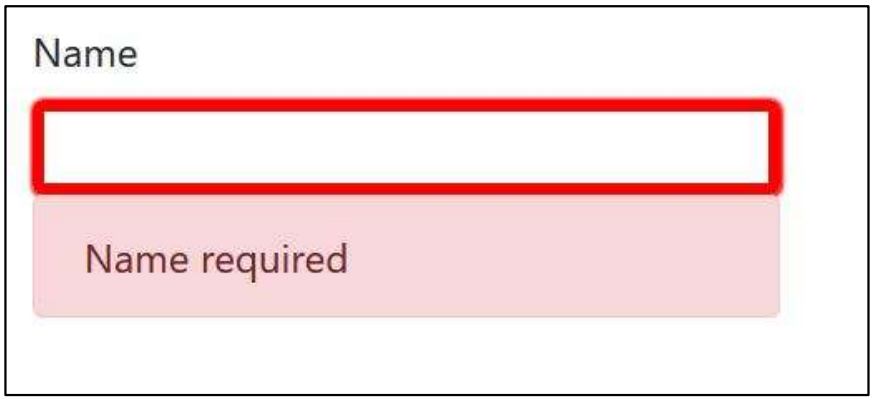
ngModel style classes

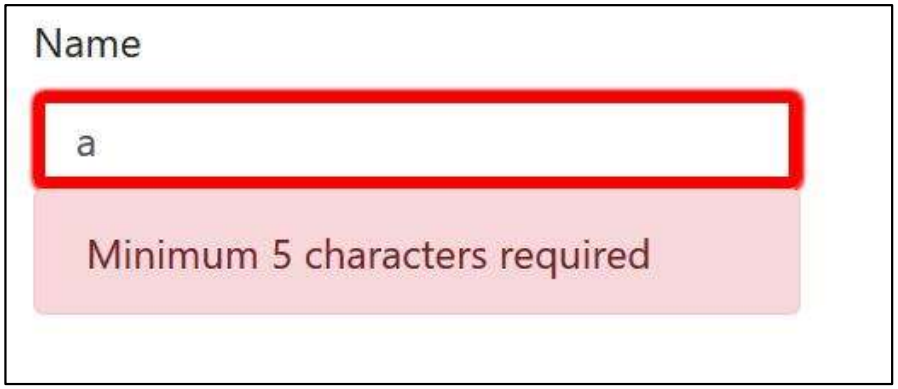
- The *ngModel* directive doesn't just track state; it updates the control with special Angular CSS classes that reflect the state
- You can leverage those class names to change the appearance of the control

State	Class if true	Class if false
The control has been visited.	ng-touched	ng-untouched
The control's value has changed.	ng-dirty	ng-pristine
The control's value is valid.	ng-valid	ng-invalid

ngModel style classes

```
.ng-touched.ng-invalid {
   border: solid 2px red;
 }
```





ngForm

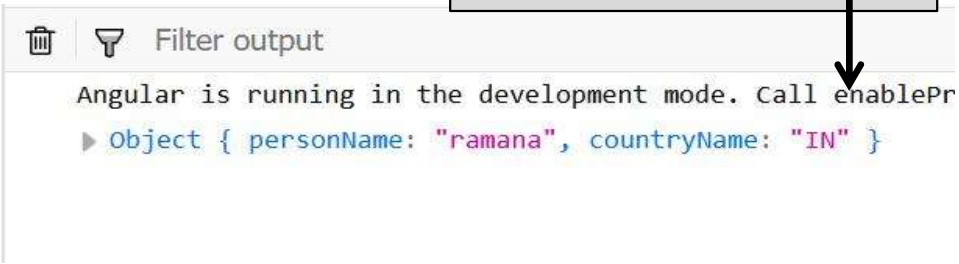
- ngForm directive is automatically applied on <form> tag
- This directive has output property ngSubmit
- ngForm creates a FormGroup object for the form
- We can access all the properties like touched, dirty, valid, invalid etc using a template name for ngForm
- value property represents an object holding values of all input elements in the group

ngForm - Example

```
<form #userForm="ngForm" (submit)=submitForm(userForm.value)>
  <div>
    <label>Name</label>
    <input ngModel name='personName' class="form-control"  #name='ngModel' />
    <label>Country</label>
    <select id='country' ngModel name='countryName' class="form-control" #country='ngModel'>
      <option *ngFor="let c of countries" [value]='c.id'>{{c.name}}</option>
    </select>
    <button type='submit' class='btn btn-primary'>Submit</button>
  </div>
</form>
```

```
countries = [
   { id: "IN", name: 'India' },
   { id: "US", name: 'U S A' },
   { id: "FR", name: 'France' }
  ];

  submitForm(data) {
   console.log(data);
  }
```



Disabling Submit for invalid Form

- As a whole it is possible to check errors for the entire form using ngForm directive
- All properties of ngModel are available in ngForm
- For example, we can disable submit button if any of the input fields are invalid

```
<form #userForm="ngForm" (submit)=submitForm(userForm.value)>

  <button type='submit' class='btn btn-primary'
            [disabled]='userForm.invalid' >Submit</button>
</form>
```

ngModelGroup

- This directive can only be used as a child of NgForm (or in other words, within <form> tags)
- Use this directive to create a sub-group within a form
- Helps to validate a sub-group of the form separately from the rest of the form
- Pass in the name you'd like this sub-group to have and it will become the key for the sub-group in the form's full value
- You can also export the directive into a local template

  ex: #myGroup="ngModelGroup"

ngModelGroup - Example

```
<form #form="ngForm" (ngSubmit)="onSubmit(form)">
    <p *ngIf="nameCtrl.invalid">Name is invalid.</p>

    <div ngModelGroup="name" #nameCtrl="ngModelGroup">
      <input name="first"  minlength="2">
      <input name="last"  required>
    </div>
</form>
```

In the code:

```
onSubmit(form){
   console.log(form.value.name.first);
   console.log(form.value.name.last);
}
```

In the template :

```
<div *ngIf="!nameCtrl.valid">
    error messages for the name group
</div>
```

Using checkbox

```
<div class='checkbox'>
  <label>
     <input type="checkbox" ngModel name="isSubscribed">
  </label>
</div>
```

Using radio group

```
<div class='radio' *ngFor="let method of contactMethods">
  <label>
    <input type="radio"  ngModel
           name="contactMethod"   [value]="method.id">
    {{method.name}}
  </label>
</div>
```

Data in the component:

```
contactMethods = [    { id: 1, name: 'email' },
                      { id: 2, name: 'phone'},
                      { id: 3, name: 'Languages' },
                  ];
```

Using dropdown list

```
<label for="country">Country</label>
 <select id='country' required   name='country'
          class="form-control" ngModel  #country='ngModel'>
     <option ></option>
     <option *ngFor="let c of countries" [value]='c.id'>{{c.name}}</option>
 </select>

 <div class='alert alert-danger' *ngIf='country.touched && country.invalid'>
      Select one country
 </div>
```

Data in the component:

```
countries = [
        { id: "IN", name: 'India' },
        { id: "US", name: 'U S A' },
        { id: "FR", name: 'France' }
 ];
```

Reactive Forms

Reactive Forms

- Reactive forms provide a model-driven approach to handling form inputs
- Instead of defining the form in the template, the structure of the form is defined in the code
- In Template Driven Forms, the FormGroup and FormControl objects are automatically created with ngForm directive
- In Reactive forms, these objects have to be created in the code and linked to the input form of the template
- To use reactive forms, we need to import the ReactiveFormsModule into our parent module

Registering ReactiveFormsModule

- import ReactiveFormsModule from the @angular/forms package and add it to your NgModule's imports array

```
import { ReactiveFormsModule } from '@angular/forms';

@NgModule({
  imports: [
    // other imports ...
    ReactiveFormsModule
  ],
})
export class AppModule { }
```

Simple Form with one field

- import ReactiveFormsModule from the @angular/forms package and add it to your NgModule's imports array

```
import { FormControl } from '@angular/forms';

@Component({
  selector: 'name-app',
  templateUrl: './name.component.html',
  styleUrls: ['./name.component.css']
})
export class NameComponent {
  name = new FormControl('Ramana');
}
```

```
<label>  Name:</label>
<input type="text" formControlName="name">
```

FormControl

- Some of the properties / methods of FormControl
  - value
  - setValue() -- setting the value directly is not possible
  - disable()
  - disabled -- boolean
  - enable()
  - enabled -- boolean
  - valid
  - dirty
  - prestine
  - …… etc

Grouping Controls

- Just as a form control instance gives control over a single input field, a form group instance tracks the form state of a group of form control instances (for example, a form).
- Each control in a form group instance is tracked by name when creating the form group

- Creating FormGroup :

```
import { FormGroup, FormControl } from '@angular/forms';

@Component({
  selector: ‘app-user’,
  - - - -
})
export class UserComponent {
     userForm = new FormGroup({
          firstName: new FormControl(‘ '),
          lastName: new FormControl(‘ '),
      });
}
```

Associating FormGroup control to View

- A form group tracks the status and changes for each of its controls
- Use formGroup attribute at form level
- Use **formControlName** for each element
- The model for the group is maintained from its members

```
<form [formGroup]=“userForm">

  <label  for=‘first> First Name: </label>
  <input  id= ‘first’ type="text" formControlName="firstName">

  <label  for=‘last> Last Name: </label>
  <input  id=‘last’ type="text" formControlName="lastName">

</form>
```

Validation with Reactive Forms

- HTML5 built-in attributes can be used for native validation, including required, minlength, and maxlength

```
<form [formGroup]=“userForm">
<div class='form-group'>
<label> First Name:</label>
<input  type="text" formControlName="firstName"  required >
</div>

<div class="alert alert-danger"
      *ngIf=“userForm.get('firstName').touched && userForm.get('firstName').invalid">
   First Name is Required
</div>

<div class='form-group'>
<label for="last"> Last Name:</label>
<input id='last' type="text" class='form-control' formControlName="lastName" >
</div>
</form>
```

Validators class

- Reactive forms include a set of validator functions which are static methods of Validators class
- Validations can be defined in the code by providing Validator functions in the FormControl object
- We can supply one function or array of functions

- Import the Validators class from the @angular/forms package

Validators class

```
import { Validators } from '@angular/forms';
…….

export class Form2Component {
   userForm: FormGroup = new FormGroup(
    {
        firstName: new FormControl(‘ ‘, [
            Validators.required,
            Validators.minLength(5)
        ]),

       lastName: new FormControl(' ‘ ,  [
             Validators.required,
             Validators.minLength(5)
       ]),
    })
}
```

Validators

```
form [formGroup]=“userForm">
  <div class='form-group'>
    <label> First Name:</label>
    <input  type="text" class='form-control' formControlName="firstName">
  </div>
  <div class="alert alert-danger"
      *ngIf=“userForm.get('firstName').touched && userForm.get('firstName').invalid">

   <div *ngIf=“userForm.get('firstName').errors.required">
     First Name is required
    </div>
    <div *ngIf=“userForm.get('firstName').errors.minlength">
     First Name should be minimum {{userForm.get('firstName').errors.minlength.requiredLength}} characters
    </div>
  </div>
  . . . . . .
</form>
```

Custom Validators

- Custom validator is a simple function which takes one input parameter of type AbstractControl
- If the validation fails, it returns an object, which contains a key-value pair
- **Key** is the name of the error and the value is always **Boolean true**.
- If the validation does not fail, it returns **null**

```
function ageRangeValidator(control: AbstractControl) {
    if (isNaN(control.value) ){
        return { ‘invalidValue': true };
    }

    If( control.value < 18 || control.value > 45)) {
        return { ‘invalidRange': true };
    }
    return null;
}
```

Custom Validators – How to use

```
loginForm = new FormGroup({
        email: new FormControl(null, [Validators.required]),
        password: new FormControl(null, [Validators.required]),
        age: new FormControl(null, [ageRangeValidator])
});
```

Custom Validators with parameters

- Passing parameters to validator function is not possible
- But it can used as a closure by defining it inside another function which takes parameters and return validator function

```
function ageRangeValidator(min: number, max: number): ValidatorFn {
        return (control: AbstractControl): { [key: string]: boolean } | null => {
            if (control.value !== undefined && (isNaN(control.value) {
                return { 'invalidValue': true };
            }

            If( control.value < 18 || control.value > 45)) {
                return { 'invalidRange': true };
            }
            return null;
        };
}
```

```
loginForm = new FormGroup({
        .......
        age: new FormControl(null, [ageRangeValidator(25, 65)])
});
```

Form Builder

- FormBuilder is the helper API to build forms in Angular.
- It provides shortcuts to create the instance of the FormControl, FormGroup or FormArray
- FormBuilder is injected into constructor

```
constructor(private formBuilder:FormBuilder){}
……————

this.contactForm = this.formBuilder.group({
        name: ['', [Validators.required, Validators.minLength(10)]],
        email: ['', [Validators.required, Validators.email]],
        gender: ['', [Validators.required]],
        country: ['', [Validators.required]],
        address: this.formBuilder.group({
                city: ['', [Validators.required]],
                street: ['', [Validators.required]],
                pincode: ['', [Validators.required]],
         })
});
```

Form Array

- FormArray is similar to FormGroup and it is used as an array that wraps around an arbitrary amount of FormControl, FormGroup or even other FormArray instances
- With **FormArray** we can add new form fields dynamically

```
angForm = new FormGroup({
        names: new FormArray([
            new FormControl('', Validators.required),
            new FormControl('', Validators.required), ])
 });
```

Routing and Navigation

Routing in S P A

- Navigation in Single page Application is different from traditional web applications
- An SPA is a web application that provides a user experience similar to a desktop application
- In an SPA, all communication with a back end occurs behind the scenes
- When a user navigates from one page to another, the page is updated dynamically without reload, even if the URL changes

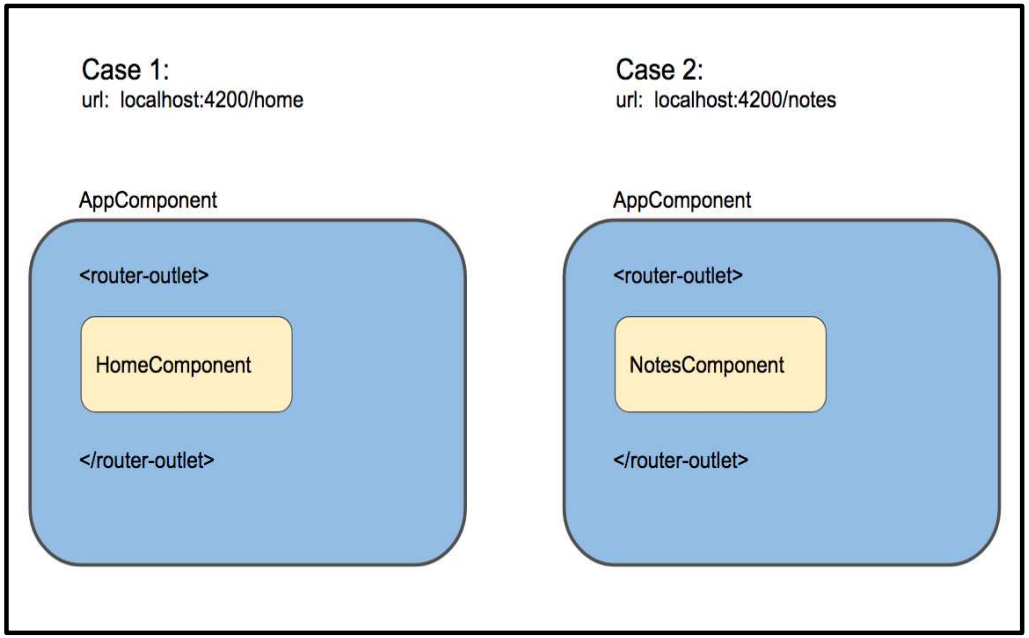
Angular Router

- It's a JavaScript router implementation that's designed to work with Angular and is packaged as **@angular/router**
- Angular Router takes care of the duties of a JavaScript router
- It activates all required Angular components to compose a page when a user navigates to a certain URL
- It lets users navigate from one page to another without page reload
- It updates the browser's history so the user can use the *back* and *forward* buttons when navigating back and forth between pages.

Routing in Angular

- Angular Router module helps in navigation in a single page

- Steps in Angular Routing:
  - Configure Routes - mapping of route with component
  - Add Router outlet - to display corresponding component
  - Add links for different routes



Routing Step 1

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';
import { CoreComponent } from './core/core.component';
const routes: Routes =
 [
      { path:'home', component:HomeComponent },
      { path:'members', component:MembersComponent }
];

@NgModule({
declarations: [  . . . .  ],
imports: [
   BrowserModule,
   RouterModule.forRoot(routes)
 ],
 providers: [],
 bootstrap: [AppComponent]
})
export class AppModule { }
```

configuring Routes

import RouterModule for top level

Routing Step 2 & 3

- While defining links we should use routerLink directive (defined RouterModule) instead of href
- href loads the whole page including all JS files which is a waste of time
- routerLink loads only the required component

```
<h3>Routing</h3>
<a routerLink='/home'>Home</a>
<a routerLink='/members'>Members</a>
<br><br>

<router-outlet></router-outlet>
```

Links for the routes

router-outlet for displaying component based on the clicked link

Wildcard routes

- Default link for ‘not found’ page can defined with wild card
- wild card route should be the last one in the list
- Wild card route will match all the routerLink references

```
const routes: Routes =
 [
        { path:'home', component:HomeComponent },
        { path:'members', component:MembersComponent },
        { path:’**’ , component:NotFoundComponent }
];
```

Route Parameters

- Suppose we have an application that displays a product list and when the user clicks on a product in the list, we want to display a page showing the detailed information about that product

- To do this you must:
  - add a route parameter ID
  - link the route to the parameter
  - add the service that reads the parameter

- Route parameters added to the route with ‘:’ symbol

Declaring Route Parameter

```
const routes: Routes = [
  { path: 'product-list', component: ProductList Component},
  { path: 'product-details/:productId', component: ProductDetailsComponent }
];
```

- **:productId** in the path of the product-details route is route parameter in the path
- For example, to see the product details page for product with ID 5, we must use the following URL:

  localhost:3000/product-details/5

Linking to Route with Parameter

- The **routerLink** directive passes an array which specifies the path and the route parameter
- **Do not use href**. Every time you click, it downloads the whole page
- routerLink loads the resources locally through javascript
- Use routerLink attribute directive for link with no parameters
- Use routerLink property attribute if link has parameters
  - First element of the array is path
  - Second clement onwards provide values for route parameter

```
<a  routerLink='products'> Get All Products </a>

<a *ngFor="let product of products"
  [routerLink]="[ 'product-details', product.id]">
  {{ product.name }}
</a>
```

Defining Route programmatically

- Use Router service of angular to navigate to a route
- Router is added in the component through dependency injection

```
Import   {Router}  from ‘@angular/router’;
// other code

constructor(private router: Router) { }
// other code

goToProductDetails(id) {
  this.router.navigate(['/product-details', id]);
}
```

Reading Route parameters

- The ProductDetailsComponent must read the parameter, then load the product based on the ID given in the parameter
- The ActivatedRoute service provides a paramMap Observable to which we can subscribe to get the route parameters
- ActivatedRoute added to the component through dependency injection

```
import { ActivatedRoute } from '@angular/router';

export class ProductDetailsComponent implements OnInit{
  selectedProduct:Product;
    constructor(private route: ActivatedRoute) { }

  ngOnInit() {
  this.route.paramMap.subscribe(params => {
      let id = +params.get('productId');
      this.selectedProduct  = .... // get theproduct using id
    });
  }
```

Using unary + to convert into number

Why ParamMap Observable

- If navigation happens from component A to Component B, Angular destroys Component A and creates Component B
- If navigation happens from Component A to Component A, it is not destroyed and onInit() method is not invoked
- In such case getting route parameter is a problem

- If we subscribe to paramMap observable, our callback method is called for each navigation

- If same component navigation is not required, we can use snapshot.paramMap which is just object

```
ngOnInit() {
  let  id = +this.route.snapshot.paramMap.get('productId')
  this.selectedProduct  = .... // get theproduct using id;
  }
```

Child routes

- When some routes may only be accessible and viewed within other routes it may be appropriate to create them as child routes.
- localhost:4200/product-details/3/overview will load OverView component

```
export const routes: Routes = [
  { path: '', redirectTo: 'product-list', pathMatch: 'full' },
  { path: 'product-list', component: ProductList },
  { path: 'product-details/:id', component: ProductDetails,
    children: [
      { path: '', redirectTo: 'overview', pathMatch: 'full' },
      { path: 'overview', component: Overview },
      { path: 'specs', component: Specs }
    ]
  }
];
```

Child routes

- Where would the components for these child routes be displayed?
- Just like we had a <router-outlet></router-outlet> for the root application component, we would have a router outlet inside the ProductDetails component

```
@Component({
  selector: 'product-details',
  template: `
    <p>Product Details: {{id}}</p>
    <nav>
      <a [routerLink]="['overview']">Overview</a>
      <a [routerLink]="['specs']">Technical Specs</a>
    </nav>
    <router-outlet></router-outlet>
})
export default class ProductDetails implements OnInit {
  id: number;
  constructor(private route: ActivatedRoute) {}
  ngOnInit() {
    this.route.params.subscribe(params => {
      this.id = +params['id']; // (+) converts string 'id' to a number
    });
  }
```

Accessing parent route parameters

- The child route component can access the parent route's parameters as follows component

```
export default class Overview {
  parentRouteId: number;
  private sub: any;
  constructor(private router: Router,  private route: ActivatedRoute) {}

  ngOnInit() {
    // Get parent ActivatedRoute of this route.
    this.sub = this.router.routerState.parent(this.route)
      .params.subscribe(params => {
        this.parentRouteId = +params["id"];
      });
  }

  ngOnDestroy() {
    this.sub.unsubscribe();
  }
}
```

Lazy Loading

- Lazy loading makes it possible to load a module when it is required
- Steps:
  - Create a module with the command

    ng g m admin –route admin –m app
  - This will create AdminModule along with its own routermodule and also creates path admin
  - It also adds route for lazy loading path for this module

```
const routes: Routes = [{ path: 'lazy', loadChildren: () =>
import('./lazy/lazy.module').then(m => m.LazyModule) }];
```

  - Do not include this module in AppModule imports

AuthGuard

- The auth guard is an angular route guard that's used to prevent unauthenticated users from accessing restricted routes
- it does this by implementing the CanActivate interface which allows the guard to decide if a route can be activated with the canActivate() method
- If the method returns true the route is activated (allowed to proceed), otherwise if the method returns false the route is blocked.
- The auth guard uses the authentication service to check if the user is logged in

AuthGuard

```
export class RouteGuardService extends AuthGuard {

  canActivate() {
    let isAuthenticated = super.canActivate();
    if (!isAuthenticated)
      return false;

    if (this.authService.currentUser.admin)
      return true;

    this.router.navigate(['/no-access']);
    return false;
  }
}
```

Router Guards to protected routes

```
const routes: Routes = [

{path:'emplist',component:EmpListComponent,canActivate:[Route
GuardService]},

  {path:'home',component:HomeComponent},

];
```